AUTONOMOUS MULTI-AGENT INVESTMENT RESEARCH SYSTEM

Code Architecture, Workflow and Implementation Report

AAI 520 — Final Project | Group 04

Team: Harsh Modi · Jaiganesh Nagabooshanam · Likhitha Potluri

Project repository: github.com/likhithapotluri/Group04_AAI520_Project

Summary

The first section explains how Group 04's AAI 520 notebook, "Autonomous Multi-Agent Investment Research & Financial Analysis System," functions and the main parts that compose it. The project automates the workflow for US-listed stock analysis, which includes research, and the creation of a Buy, Sell or Hold recommendation backed by research data, reports on company filings, and news, to name a few.

The starting point of this architecture includes a research planner followed by a supervisor router, and finally three specialized agents. There is also a news prompt chain, a report writer, an evaluator, and self-reflection checks. Long term memory is persistent across sessions and implemented using SQLite. There is a central tool registry that manages caching, data access, and provides error isolation and logging for all of the tools. The notebook's final section demonstrates successful runs of the system using NVDA and AAPL.

The notebook also claims that both systems passed the evaluation's quality checks on their very first try. The documented results of the system in the notebook are not independent assessments of investment systems and are not predictions of system accuracy and future performance.

1. Objectives

The objective is to build an end to end research workflow from company input through evidence collection, specialist analysis, synthesis, evaluation and report.

The workflow is designed and is looked at reducing the repetitive manual research while making planning, intermediate decisions and the relevant data gaps traceable.

Workflow steps:

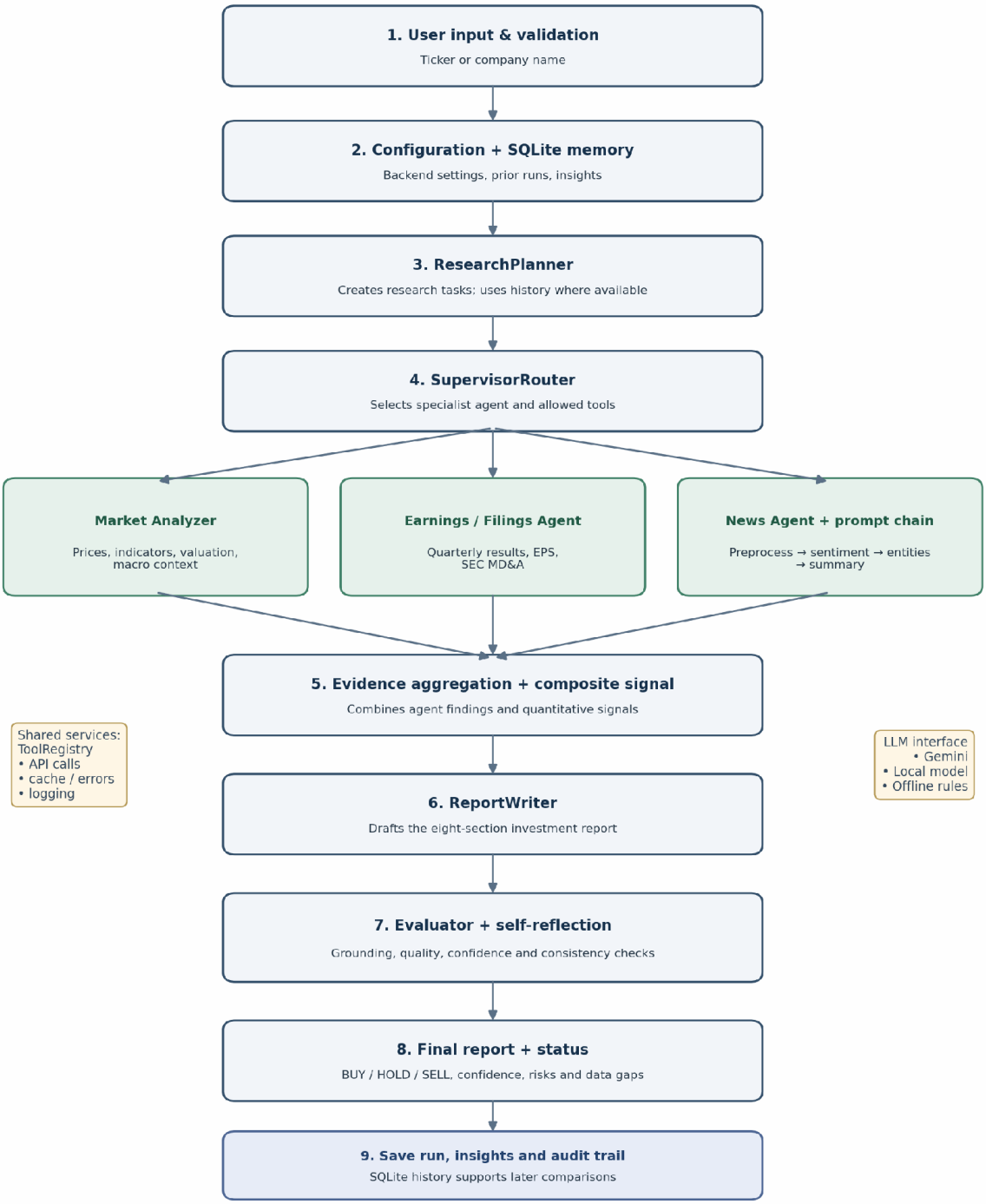
1. Input: The Input fed into the Agentic workflow - an US-listed company name ot ticker, resolved by the input-validation logic.
2. Output: a research report with a predicted recommendation, supporting with confidence, supporting with evidence, any visible risks and observed data gaps.
3. Workflow capabilities: Using the relevant prompt chaining - autonomous task decomposition, dynamic tool orchestration, specialist routing, iterative evaluation and use the persistent memory.
4. Scope limitation: Not a financial-advice service instead an educational exercise.

2. Architecture

The workflow system can be split as nine connected stages as perceived in the following flow:

1. Input and validation — accept either a ticker (eg., MSFT) or company name (eg., Microsoft).
2. Configuration and memory — select a backend (CLI) and retrieve any earlier runs/insights.
3. Research planning — decompose the requested investment question into the planned research tasks.
4. Supervisor routing — assign each task to the related specialist and allowed tools.
5. Specialist analysis — as required run market, earnings/filings and news analysis.
6. Evidence aggregation — combine agent findings and quantitative signals.
7. Report drafting — produce the required report sections.
8. Evaluation and reflection — check grounding, completeness, confidence and signal consistency; revise when needed.
9. Persistence and inspection — save the run and insights, then expose history and dashboard visualizations.

Figure 1 shows the overall architecture. The LLM interface and ToolRegistry are shared services used across the workflow.



3. Core Software Components

Component	Role - Why do we need them ?
Config / backend selection	Keeps model and runtime choices configurable.
LLM interface	Agents can share a backend contract.
MemoryStore	Supports traceability and comparisons across runs.
ToolRegistry	Makes tool access controlled and reusable.

ResearchPlanner	Turns a broad goal into actionable tasks.
SupervisorRouter	Routes work without giving agents unrestricted tool access.
MarketAnalyzerAgent	Builds the market view.
EarningsFilingsAgent	Builds the financial-performance view.
NewsAgent / NewsPromptChain	Converts unstructured headlines into structured evidence.
ReportWriter	Synthesizes the specialist outputs.
EvaluatorAgent / SelfReflection	Provides a quality gate before finalization.
InvestmentResearchOrchestrator	Acts as the central controller.

4. Data Sources and Tool Access

Tool	Source
MarketDataTool	Yahoo Finance via yfinance
EarningsTool	Yahoo Finance via yfinance
MacroTool	FRED
NewsTool	NewsAPI with Yahoo fallback
SECFilingsTool	SEC EDGAR

The ToolRegistry has the responsibility for calling the relevant tools by the name. It does the caching tool outputs for a configured period (12 hours by default is set in the notebook) and records any accounted failures as related data gaps rather than failing it together. The notebook also identifies a limitation: error-carrying results might be cached unless caching is explicitly prevented..

5. The Three AI Workflow Patterns

5.1 Prompt Chaining — News Pipeline

The NewsPromptChain passes the article information through a sequence of steps. It first preprocesses the raw articles and then it scores the sentiments, consecutively extracts the entities and the included events. Once those steps are done, they creates a very concise summary. Each stage uses the previous stage's output.

Stage	Purpose	Fallback steps listed in the notebook
Preprocess	Remove noise, duplicates and removed-content placeholders.	Python preprocessing.
Sentiment	Assign a sentiment label, score and relevance.	FinBERT where available; word lists if it cannot load.
Entities and events	Identify organizations, people, products and event signals.	Capitalized-phrase detection and keyword rules.
Summary	Summarize catalysts, risks and the overall news signal.	Template based on sentiment statistics and top events.

The aggregate news sentiment is contextually weighted and then calculated. The pipeline includes checks for empty input, missing labels, score ranges and valid labels. If no news is returned, then the system records a data gap.

5.2 Specialist Routing — Planner and Supervisor

The Research Planner breaks the request into four to seven tasks, depending on the research needed. When available, the LLM can also use the previous run to identify what has changed. If the LLM is unavailable, the planner relies on predefined rules the configurator had set up instead.
The Supervisor Router assigns each task to a suitable specialist and limits the available tools to those relevant to that role. If the LLM fails or the system is offline, keyword matching is used to decide where each task should go. The workflow also makes sure no specialist is left out.
The process starts with Market analysis, followed by Earnings and Filings, and then News. Starting with the market helps establish which company is being researched, making it easier to find relevant news later. Tasks that fall under the same specialist are combined where possible, so the system doesn't repeat the same work unnecessarily.

5.3 Evaluator–Optimizer and Self-Reflection

The ReportWriter prepares the report, which is then checked for quality and accuracy. If it doesn't meet the defined standards, then the system reviews and revises it and runs the checks again, provided that there are related evaluation rounds remaining. The notebook allows up to three rounds of evaluation before the process stops.

Check	Purpose
Evaluator score	Scores factual grounding, hallucination avoidance, coherence, completeness and risk balance.
Numeric grounding	Estimates how much of the report's numerical content can be traced to supplied evidence.
Signal consistency	Checks alignment between the written recommendation and the composite quantitative signal.
Required sections	Checks for Executive Summary, Market & Technicals, Fundamentals & Earnings, News & Sentiment, Macro Context, Bull vs Bear, Risks and Recommendation.
Critical issues / confidence	Prevents a high aggregate score from being the only condition for approval.

The notebook calculates the quality score using three factors: evaluator score (60%), numeric grounding (25%) and signal consistency (15%). To succeed, the report requires a score of at least 0.80, at the confidence of 0.65 or higher, no high or critical issues and all the required sections in place. If none of the LLM-generated drafts passes, the system tries a rule-based report and evaluates that as well. If the final report still falls short, it is marked LOW_CONFIDENCE_REVIEW for a human to check.

6. Composite Signal and Recommendation

The system brings together market indicators such as trend, momentum and RSI with news sentiment and signals from specialist agents. These inputs are combined into a single score ranging from −1 to +1. In rule-based mode, a score above 0.2 results in a BUY recommendation, while a score below −0.2 results in SELL. Scores between these limits are classified as HOLD. The confidence score depends on how closely the signals agree and how much data is available, with a maximum of 0.95.
The thresholds are set in the notebook and have not been validated as investment rules. The recommendations are meant to summarise the available evidence for learning and analysis.

7. Persistent Memory and Run Lifecycle

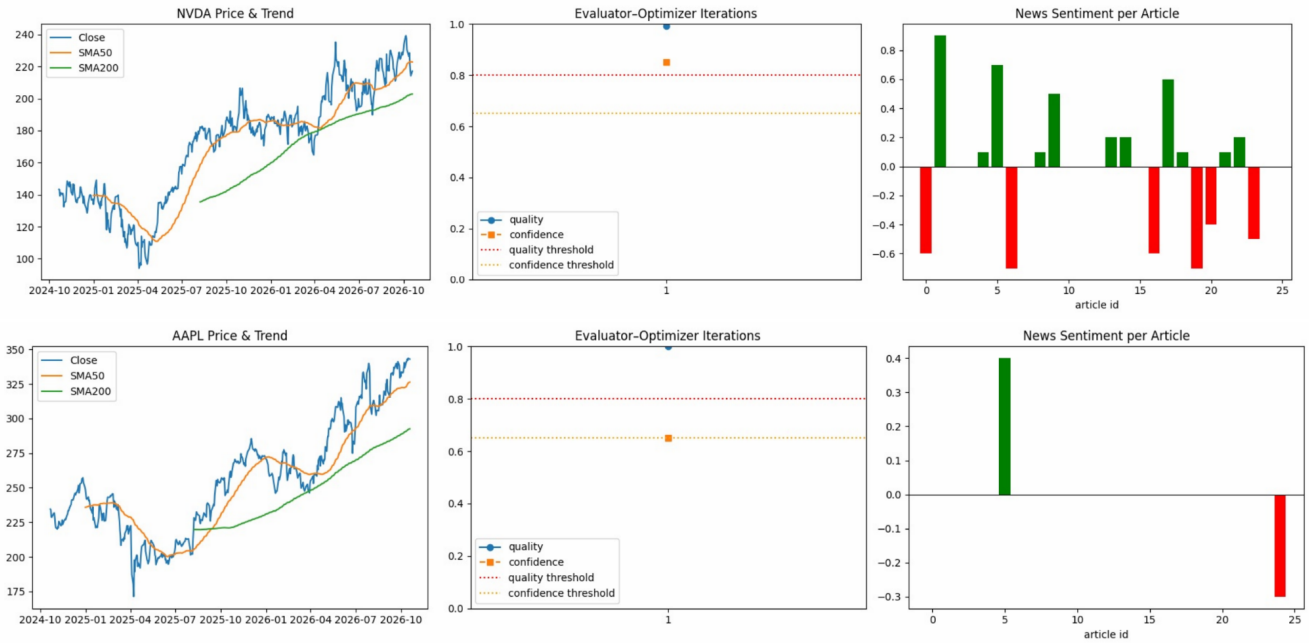
SQLite table	Stored information	Purpose
runs	Ticker, date, price, recommendation, confidence, status and report.	Review run history and compare past calls.
insights	Agent or outcome-tracker notes.	Provide prior context to future planning and writing.

scratchpad	Plan, routing, analysis and evaluation-round notes.	Audit trail and debugging.
data_cache	Timestamped tool results.	Avoid unnecessary repeat calls within the cache lifetime.

Each run gets its own unique ID. The orchestrator retrieves relevant memory, plans the research, assigns tasks to the agents and runs them. Once the results are reported and collected, it calculates the necessary composite signal, then it prepares the report and then checks its quality before saving the report and key insights. If a run fails, the status is updated to FAILED so that any incomplete runs or failed runs are clearly identified.

8. Notebook Execution and Dashboard

Notebook area	Action
Setup and configuration	Checks packages, loads imports, settings and backend selection.
Memory initialization	Mounts Google Drive when available and opens the SQLite database.
NVDA demonstration	Runs the orchestrator for NVIDIA and prints recommendation, confidence, status, routing and report.
Cache clearing	Clears the data_cache table before the subsequent demonstration.
AAPL demonstration	Runs the orchestrator for Apple after the cache is cleared.
Dashboard	Plots close price with SMA-50/SMA-200, evaluator quality/confidence across rounds and per-article news sentiment.
Memory inspection	Displays run history, recalled insights and evaluation history.



9. Results Reported by the Notebook

The notebook's conclusion reports the following observations from its demonstration runs.

Reported result	NVDA run	AAPL run
LLM calls	13	27
Rule-based fallbacks	0	0
Routing method	All five routing decisions used LLM routing	All six routing decisions used LLM routing
Evaluator quality	0.993	1.000
Confidence	0.85	0.65
Evaluation rounds to pass	1	1
Elapsed runtime (approx.)	4 min 30 sec	2 min 26 sec

The notebook documents that clearing the cache fixed an issue where no news articles appeared. On the next AAPL run, it retrieved and analyzed 25 articles, and the News & Sentiment section was populated. This shows that caching can reduce repeated work, but stale cached data may also hide changes in source availability or configuration.

10. Recommended Future Improvements

There are certain areas that are worth improving in the current implementation. The cache should ignore results that contain errors, and it would help to keep track of when the data was fetched and whether it came from the cache. The recommendations would also need to be tested against what actually happened to the stock price in the past over time, including the transaction costs and comparison with a market benchmark. Confidence scores would be advised to be reviewed in the same way to see whether they reflect the results in the actual practice. Further improvement would be to use a separate model to review the generated report rather than asking the same model to assess its own work. Adding semantic search could help retrieve older filings, earnings calls and news when they are relevant to the current analysis. The system could also include specialists for peer comparisons, ESG and insider trading, depending on the data available and the permissions granted.
Testing should cover situations such as empty responses, invalid model output, API limits, incorrect ticker identification, stale cache entries and repeated executions. Finally, the prompts, model versions, source data and scoring rules should be recorded for each run. This would make it easier to trace a result back to the inputs and understand why two runs produced different reports.

11. Conclusion

The notebook shows how different agents work together to carry out investment research, from collecting market information to generating the final report. The orchestrator controls the flow between these components, including task assignment, tool selection and handling failures when a model service is unavailable. It also maintains run history and checks the report before it is returned. Reading the orchestrator first gives a clear picture of how the different parts connect together to form the required workflow.
The current implementation is a really good starting point, but the recommendations need to be tested against actual market performance before they can be relied on. Source data needs to be verified, and the evaluation rules need to be checked to see whether they identify incorrect or unsupported conclusions. Backtesting past the recommendations and recording the cases where the data availability is insufficient would help assess how well the system performs. A report passing all the required quality checks only confirms that it meets the focused conditions that are defined in the notebook. Whether the recommendation is right is altogether a separate question.

References:

Project notebook: AAI_520_Final_Project_Group_04.ipynb.

Project repository: http://github.com/likhithapotluri/Group04_AAI520_Project.

Data-source interfaces referenced in the notebook: Yahoo Finance/yfinance, FRED, SEC EDGAR and NewsAPI.

Model/backend options described in the notebook: Google Gemini, Qwen2.5-1.5B-Instruct local model and offline rules.